Mixlist

Miranda Secaur, Miranda Shoraji, Mariam Yassa, Judy Nashwati, Faculty Advisor: Amartya Sen

Department of Computer Science | Oakland University | sen@oakland.edu

INTRODUCTION / MOTIVATION

What is the problem?

The problem that Mixlist aims to address is the lack of access to a tool that can generate music for a group of people that will take each of their music tastes into account while letting the host maintain control of the genre of music. Current solutions rely on blending a groups music taste into an unstructured list of songs theyn have in common, otherwise one person usually selects music or a predetermined playlist. While these solutions will do the job, they may not match the tastes of everyone involved. This is especially challenging in environments such as businesses, social gatherings, or shared spaces where multiple people are listening together.

This tool is important because music directly influences user experience and behavior, yet there is no simple way to tailor playlists to a group's collective preferences. Our team was motivated by the gap in existing tools, such as Spotify Blend, which often include songs or genres that not all users enjoy. This affects both individuals and businesses that want to create an environment that better reflects their audience.

Why does it matter?

Solving this problem improves both user experience and real-world outcomes. Studies and observations show that people are more engaged and spend more time in environments where the music aligns with their preferences. However, businesses and users currently lack a scalable way to determine and generate this type of music efficiently.

This project matters because it introduces a more personalized and collaborative approach to music selection, reducing the time and effort required to create playlists while improving satisfaction for all listeners. By addressing the limitations of existing systems, Mixlist has the potential to enhance social experiences, support small businesses, and create more inclusive listening environments.

Research / Design Questions:

- *RQ1: How can multiple users music preferences be combined into a single playlist that reflects group taste while remaining balanced?*

Multiple users preferences can be combined by collecting listening data and assigning weights to songs based on shared patterns and overall group interest. This helps create a playlist that reflects everyone's taste instead of favoring just one user, and maintains consistency through genre filtering.

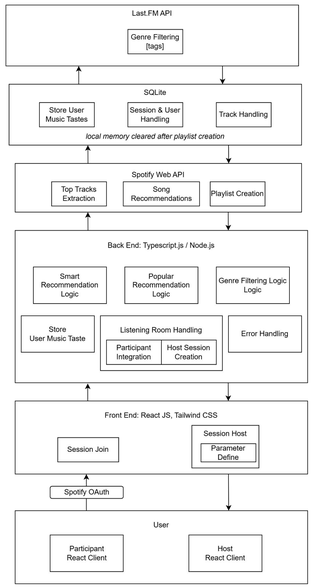
- *RQ2: How can a system support multiple users joining and interacting within a shared session while maintaining performance and stability?*

The system can support multiple users by using a session-based design where people can join and their music taste is added to the existibng session, keeping the process efficient and limiting unnecessary data requests helps maintain performance and stability even as more users join.

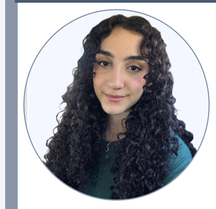
OBJECTIVES

- Objective 1: Develop a system that generates a shared playlist by combining multiple users' music preferences into a balanced group profile refined with genre filtering.
- Objective 2: Design a session-based platform that allows users to create and join Mixlist sessions using a room code.
- Objective 3: Implement playlist generation logic that produces consistent and repeatable results based on user preferences and host settings.
- Objective 4: Ensure system performance remains efficient and stable under multiple users and repeated interactions.

SYSTEM ARCHITECTURE / DIAGRAM



TEAM & ACKNOWLEDGMENTS



Miranda Shoraji

Database, Backend, UX/UI Dev / Senior

Miranda Secaur

Algorithm Design, Backend, UX/UI Dev / Senior

Judy Nashwati

UX/UI Design/ Senior

Mariam Yassa

UX/UI Desgin / Senior

Acknowledgments

We would like to thank Dr. Sen for his guidance and support throughout this project. We also thank our peers for their feedback during development and testing. This project was completed as part of the Senior Capstone course, and utilizes the Spotify Web API for music data integration.

METHODOLOGY / APPROACH

01 Data Collection / Input

Users' current top tracks are pulled from Spotify AI and appended to a SQLite database. Then, artist information, specifically genre tags, are extracted through Last.FM calls and written to the database. The host's genre filter is a predetermined set of options that the developers have chosen.

02 System Design / Architecture

The user and host interact with the application through the React client, and tracks and access tokens are pulled through Spotify OAuth. The frontend guides the user through session creation and playlist generation. The backend applies playlist generation logic and manages the users in each session. Spotify API returns users' top tracks and optionally song recommendations. Last.fm API calls provide genre information for playlist generation logic. The SQLite database store user track data, manages sessions and users, and handles tracks.

03 Core Algorithm / Model

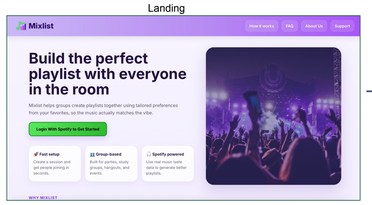
Mixlist's key algorithm is its playlist generation logic. The algorithm pulls from track data collected from users as they join the session. In the algorithm, additional information about provided user tracks are provided by Last.fm calls. Once the genre of each song in the database is known, tracks are filtered to fit the host's parameters. Then, Spotify calls are utilized to add recommendations to fill a playlist that may be otherwise lacking in tracks.
This approach utilizes the strengths of Spotify recommendation logic and fills the gap that Spotify API has with providing genre information. The backend logic then takes into account Host preferences of only user data or user data plus recommendations to provide a playlist that users have a say in while maintaining consistent musically.

04 Implementation & Tools

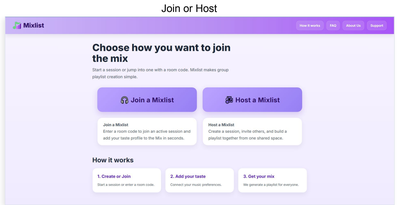
Mixlist uses React, Typescript, and Vite for frontend components, and Typescript and JSON for backend logic. The database is SQLite as we do not need persistent data outside of the users currently using the application. The application is deployed on Render for quick and easy access, as well as automatic updates through a commit on the GitHub main branch. Mixlist calls the Spotify API for taste profile data, and Spotify OAuth for authorizing a user's Spotify data to our applicaiton. It aso implements the LastFM API for genre handling in our algorithm. The application is deployed and hosted on Render, with code coming from the most recent github commit.

DISCUSSION & DEMO

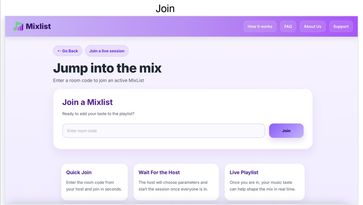
Landing



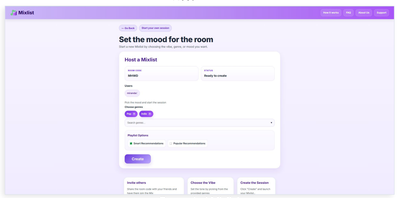
Join or Host



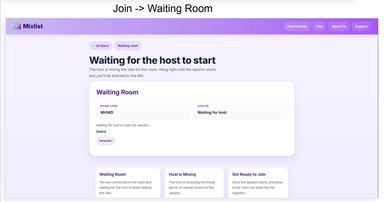
Join



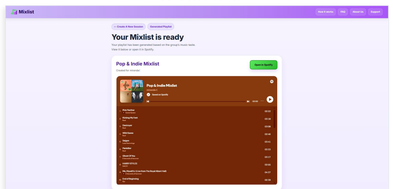
Host



Join -> Waiting Room



Generated Mixlist



Our results show that combining multiple users listening data can generate a playlist that better reflects the group's overall preferences rather than just one person's taste. Performance met expectations, especially because of how smoothly the system blended different music styles. It was surprising how opposing music tastes can create a dynamic and interesting final playlist. Compared to a baseline of single-user playlists, Mixlist produced more balanced and diverse results, making it a essential musical tool for group settings.

SOFTWARE BILL OF MATERIALS (SBOM)

Languages:	*Typescript, JavaScript, HMTL*
Frameworks / Libraries:	*React, Node.js, Express, Tailwind CSS*
APIs / Services:	*Spotify Web API, SpotifyOAuth, Last.fm API*
Database:	*SQLite*
DevOps / Infrastructure:	*Render (deployment), GitHub (version control)*
Tools:	*Vite, Axios, ts-node-dev, VS Code, npm, Github Desktop*

TESTING, RESULTS & EVALUATION

100%	3140ms	5
Accuracy *on test set*	**Latency** *avg response*	**Users Tested** *participants*

UNIT TESTING

Feature Tested	Status
Spotify Authentication	✅
Token Exchange	✅
Session Creation	✅
Listener Join/Leave	✅
Taste Aggregation	✅
Playlist Generation	✅

INTEGRATION TESTING

OAuth Testing Goals:	Session Testing Goals:	Listener Testing Goals:	Aggregation Goals:	DB Testing Goals:
Is a user able to authenticate using OAuth?	*Is a host able to create a mixlist room?*	*Is a user able to join a Mixlist session?*	*Does a playlist accurately return music data?*	*Is the session cleared upon playlist creation?*
Is user music data returned upon authentication?	*Is the database updated to reflect session users?*	*Is music data returned when user joins?*	*Is the returned playlist following parameters?*	*Are multiple sessions supported?*

Figure 1: Testing plan throughout development. Once goals were accomplished, moved on to the next phase of testing.

System Performance

- Latency for generating a mixlist is built in to be 3000ms plus the time it takes to load the page, because Mixlist generates a playlist faster than Spotify API has a chance to add the tracks after the logc is executed. While this latency is not impressive by any means, it is necessary in order to produce consistent results.
- The database is cleared after 6 seconds, meaning that zero user data is stored after playlist generation. This timeframe gives the backend enough time to grab necessary data (tracks, user list, genre), with room to scale as more users add tracks to the database.

Playlist Quality

- Through testing, users report that a generated Mixlist generally follows the parameters set by the host, creating a cohesive playlist of a given genre.
- Because of the way genre tags work, there are some instances of songs appearing in a Mixlist that do not belong.

Algorithm Performance

- Users testing the app report that "smart track" recommended tracks are accurate to the subgenres of a given mixlist, while "popular track" recommendations result in accurate recommendations and return tracks that are considered more popular.
- Through testing users report that multiple music tastes are taken into account in playlist generation, with one possible downside being that if a given user does not listen to the genre the host as chosen, little to no tracks from that user will appear.

User Experience

- Listeners are able to join a mixlist in 2 clicks and typing in a 4 letter room code, and a Host can set up a room in 4 clicks and choosing a genre, emphasizing ease of use.

System Reliability

- Under normal use and while using the app as expected, the only time that the application will have trouble is if the host selects a genre that no users listen to, and does not choose to have recommendations, which will return a Mixlist of zero songs. While this Mixlist would be useless to the group, it would be expected behavior for the logic that the user is asking the application to preform.

Scalability & Limitations

- Scalability is limited by the restrictions of Spotify API. For a host to be able to host a room of more than 5 users at a time, they would need to ask Spotify for a development account from a business email, which was not able to be done at time of development. Once out of development mode, Mixlist should not have issues with hosting more than 5 users.

Cost Effectiveness

- The cost of running Mixlist is close to zero when running the application on development/free tier API & deployment software. Further scaling would result in scaled costs. When hosted locally cost would be zero.

CONCLUSIONS & FUTURE WORK

Conclusions

- **Real Time Collaboration:** Mixlist demonstrates that a collaborative playlist can be generated in seconds by combining multiple users' music preferences while maintaining a stable and responsive session experience. Throughout development, it became clear that effective session management and data synchronization are essential for building a strong multi-user application, requiring careful handling of state and timing to ensure consistency across users.
- **Balanced Recommendations:** The implementation of an algorithm with a weighted aggregation approach shows that group-based music recommendations can be balanced effectively, allowing the playlist to refelect an equally shared pool of preferences without being dominated by one user, and allowing for some recommendations as needed.
- **Practical Use Case**: The project validates that collaborative music selection is a viable alternative to individual recommendation systems. This can positively impact any social environment such as group settings, events, and shared public spaces.

Future Work

- Further enhance the playlist generation algorithm by improving how track weights are calculated, especially in cases where user preferences conflict or limited data is available in genres not listened to by a user, and implement real time updates to the playlist after generation.
- Enhance the user experience by improving the interface flow, and ensure a smoother experience across devices.
- Expand the system with additional features such as allowing users to revisit a session, and introducing optional recommendation toggles (e.g. surprise me).
- Implement user participation through apple music and other music platforms

REFERENCES

- Baltrunas, L., Makcinskas, T., & Ricci, F. (2010). Group recommendations with rank aggregation and collaborative filtering.
- Last.fm. (2024). Last.fm API documentation. https://www.last.fm/api
- Programming with Mosh. (2023, March 12). React tutorial for beginners [Video]. YouTube. https://www.youtube.com/watch?v=SqcY0GlETPk
- Ricci, F., Rokach, L., & Shapira, B. (2015). Recommender systems handbook. Springer.
- Spotify. (2023). Blend feature usage statistics. https://newsroom.spotify.com/
- Spotify. (2024). Spotify Web API documentation. https://developer.spotify.com/documentation/

Contact: {mirandashoraji@oakland.edu} {mirandasecaur@oakland.edu} {jnashwati@oakland.edu} {myassa2@oakland.edu} | https://github.com/mirandashor/Mixlist-Senior-Capstone